Algorithm Design — Course Project

0/1 Knapsack:

Dynamic Programming vs. Greedy vs. Machine Learning

Jiacheng Cao

Part 1 of 2 — Knapsack Segment

The 0/1 Knapsack Problem

A classic NP-hard combinatorial optimization problem

Definition

Given n items, each with weight w_i and value v_i, and a knapsack with capacity W.

Goal: Select a subset of items to maximize total value without exceeding capacity.

Constraint: Each item is fully included or excluded — no fractions (0/1).

Formal statement

$$\text{maximize} \quad \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

Why it matters

- Resource allocation under budget caps
- Investment portfolio selection
- Cargo & container loading
- Cutting stock & bin packing
- Project scheduling under constraints

NP-hard in general.
Pseudo-polynomial DP $O(n \cdot W)$ solves moderate cases exactly.

Example Instance (W = 15)

A 6-item instance used throughout the presentation

Item	Weight	Value	Ratio (v/w)
A	3	8	2.67
B	4	9	2.25
C	5	10	2.00
D	8	20	2.50
E	7	19	**2.71**
F	2	3	1.50

Key observation

Item E has the best v/w ratio (2.71).

But the optimal solution is **D + E**:

- weight = 8 + 7 = 15 (exact fit)
- value = 20 + 19 = 39

Greedy will miss this.

Dynamic Programming — Bottom-Up Table Fill

$$dp[i][w] = \max\big(dp[i-1][w],\ dp[i-1][w-w_i] + v_i\big)$$

	w=0	w=7	w=8	w=10	w=12	w=15
base	0	0	0	0	0	0
i=1 (A)	0	8	8	8	8	8
i=2 (B)	0	17	17	17	17	17
i=3 (C)	0	17	18	19	27	27
i=4 (D)	0	17	20	20	29	37
i=5 (E)	0	19	20	27	29	**39**
i=6 (F)	0	19	20	27	30	**39**

Complexity

O(n × W)

cells: n·W
= 6·15 = 90

Answer:

dp[6][15] = 39

(optimal value)

DP Traceback — Recovering the Solution

Walk backwards from dp[6][15]; if a cell differs from the row above, the item was taken

Step	Item	dp[i][w] vs dp[i−1][w]	Decision	Remaining W
1	**F**	39 = 39	Skip	15
2	**E**	39 ≠ 37	**Take**	15 → 8
3	**D**	20 ≠ 18	**Take**	8 → 0
4	**C**	0 = 0	Skip	0
5	**B**	0 = 0	Skip	0
6	**A**	0 = 0	Skip	0

Optimal solution

Items selected:

D + E

Weight:

15 / 15

Value:

39 (optimal)

Rule: *$dp[i][w] > dp[i-1][w] \Rightarrow$ item i was taken; update $w \leftarrow w - w_i$.*

Greedy Heuristic — Sort by Value/Weight Ratio

Step	Item	Ratio	Action	Remaining
1	**E**	2.71	**Take**	15 → 8
2	**A**	2.67	**Take**	8 → 5
3	**D**	2.50	Skip (w=8 > 5)	5
4	**B**	2.25	**Take**	5 → 1
5	**C**	2.00	Skip (w=5 > 1)	1
6	**F**	1.50	Skip (w=2 > 1)	1

Result: E, A, B | weight 14/15 | value **36**

Optimality gap

7.7%

36 vs 39 (DP)

Why greedy fails on 0/1

- Greedy grabs E, A, and B by ratio and blocks item D.
- D + E together fill the capacity exactly for the value 39.

*Greedy **is** optimal for the fractional variant — not 0/1.*

ML Approach — Learning to Solve Knapsack

Per-item binary classification trained on DP-labeled instances

Generate 2,000 random instances → **Solve each with DP (ground truth)** → **Extract 8 features per item** → **Train Random Forest + MLP**

Training pipeline ↑

New instance → **Extract features** → **Predict P(include)** → **Feasibility correction** → **Solution**

Inference pipeline ↑

Metric	Random Forest	MLP
Accuracy	91.9%	92.4%
Training time	0.5 s	66 s
Train / test	32,000 / 8,000	32,000 / 8,000
Split	Instance-level (no leakage)	Instance-level (no leakage)

ML Predictions on the Example Instance

Item	W	V	RF P	RF Init	After corr.	MLP P	MLP Init	After corr.	DP Truth
A	3	8	0.80	Include	**Include**	0.86	Include	Dropped	Exclude
B	4	9	0.54	Include	Dropped	0.79	Include	Dropped	Exclude
C	5	10	0.38	Exclude	Exclude	0.61	Include	Dropped	Exclude
D	8	20	0.67	Include	Dropped	1.00	Include	**Include**	**Include**
E	7	19	0.82	Include	**Include**	1.00	Include	**Include**	**Include**
F	2	3	0.43	Exclude	Exclude	0.26	Exclude	Exclude	Exclude

RF feasibility correction

Initial: {A, B, D, E}, w = 22 > 15
Drop B (conf 0.54) → w = 18
Drop D (conf 0.67) → w = 10

Final: {A, E} | value **27** | gap **30.8%**

MLP feasibility correction

Initial: {A, B, C, D, E}, w = 27 > 15
Drop C (conf 0.61) → w = 22
Drop B (conf 0.79) → w = 18
Drop A (conf 0.86) → w = 15

Final: {D, E} | value **39** | gap **0.0% (optimal)**

Experimental Setup

Three test families × two size tiers × four solvers

1 Uncorrelated Random

Weights and values drawn independently. Baseline random-instance behavior.

2 Correlated

Values positively correlated with weights. Harder for ratio-based heuristics.

3 Adversarial

Hand-crafted bait items with inflated v/w ratio. Designed to break greedy.

Parameters & metrics

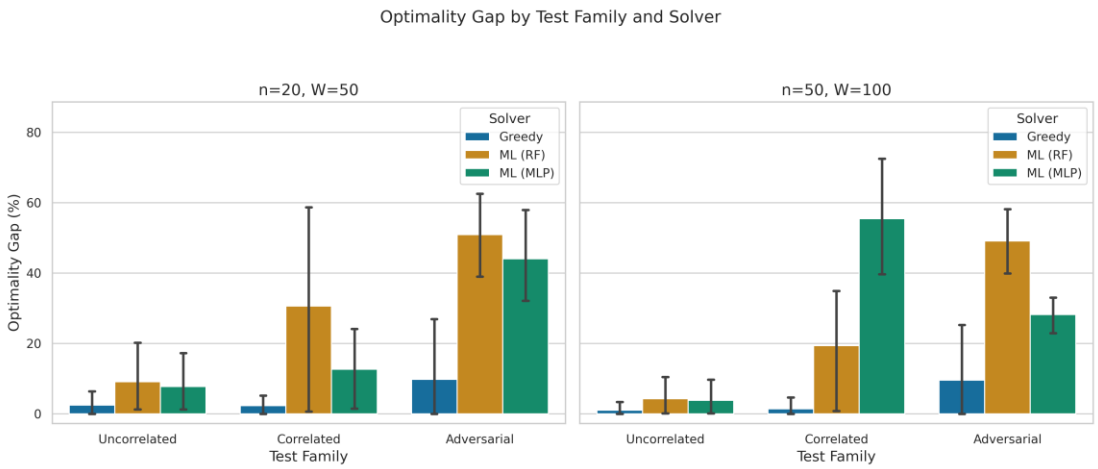
Size tiers: n=20 / W=50 and n=50 / W=100 • **Instances:** 10 per family per tier

Solvers: DP (exact), Greedy, ML (RF), ML (MLP)

Metrics: optimality gap (%) and runtime (s)

Note: *ML models trained only on uncorrelated n=20, W=50 instances — correlated/adversarial test generalization.*

Results — Optimality Gap



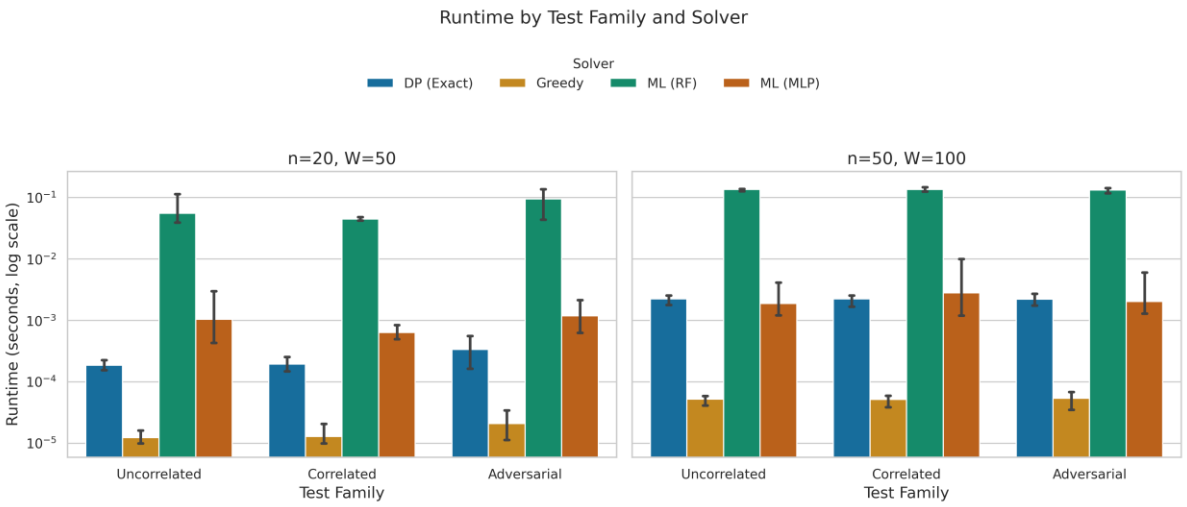
Family	n	Greedy	RF	MLP
Uncorr.	20	2.5%	9.2%	7.8%
Correlated	20	2.4%	30.7%	12.7%
Adversarial	20	9.8%	51.0%	44.1%
Uncorr.	50	1.1%	4.3%	3.9%
Correlated	50	1.5%	19.5%	**55.5%**
Adversarial	50	9.6%	49.2%	28.3%

Takeaways

- DP is always optimal (0% by definition)
- Greedy: ~1–3% on random, ~10% on adversarial
- ML is distribution-sensitive — fails on OOD
- MLP hits 55.5% mean gap on correlated n=50 (10 instances, high variance)

Results — Runtime

Warm inference (ML models preloaded), log scale



Observed ranges

Greedy
10–50 µs · O(n log n)

DP
0.2–2.3 ms · O(n·W)

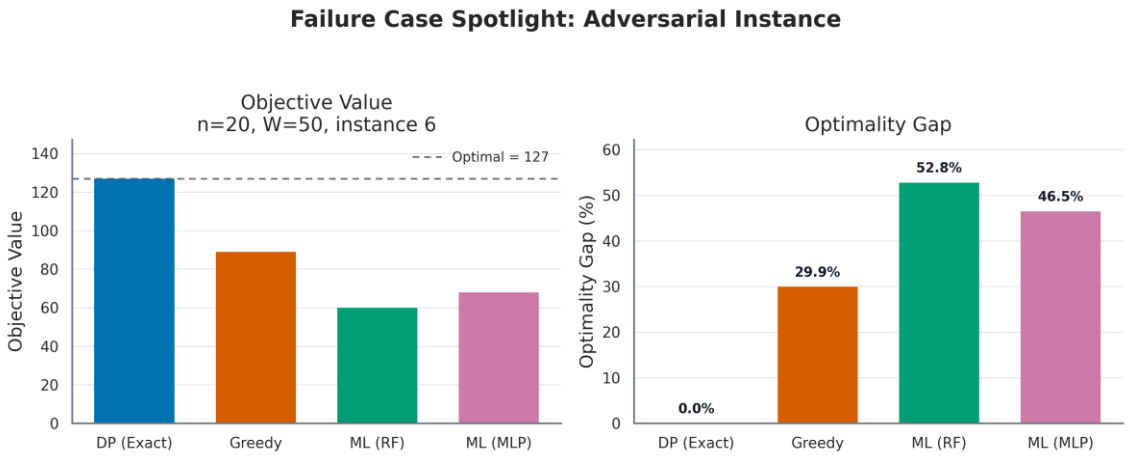
ML (MLP)
0.5–4 ms · single pass

ML (RF)
40–135 ms · 200-tree query

At these sizes runtime is not the bottleneck; at larger n·W DP would dominate.

Failure Spotlight — When Does Each Approach Fail?

Worst-case adversarial instance — primarily spotlighting Greedy



Solver	Fails when…	Root cause
DP	Never (quality)	Exhaustive, but O(n·W) scales poorly
Greedy	High-ratio 'bait' blocks better combos	Local optimum ≠ global optimum
ML (RF)	Out-of-distribution inputs	Trained patterns don't generalize
ML (MLP)	OOD + miscalibrated confidence	Wrong items dropped in feasibility fix

Note: *Greedy's mean adversarial gap is ~10%; this instance reaches ~30%. RF/MLP here (52.8% / 46.5%) track their family means (51.0% / 44.1%).*

The tradeoff: *DP guarantees optimality but scales poorly. Greedy is fast, but structurally myopic. ML learns patterns but needs representative training data.*

0/1 Knapsack — Key Takeaways

Aspect	DP	Greedy	ML
Solution quality	Optimal (exact)	Good on random (~1–3%)	Variable (distribution-dependent)
Runtime	O(n·W) pseudo-poly	O(n log n)	Inference + model overhead
Scalability	Limited by W	Excellent	Good once trained
Reliability	Always correct	Consistent heuristic	Needs representative data
Best for	Small–medium, optimality needed	Fast approximation when ratio fits	Problems with learnable structure

Key insights

1. Greedy is a surprisingly effective heuristic for 0/1 Knapsack in practice.
2. ML can mimic DP but is distribution-dependent — it fails on out-of-distribution inputs.
3. Feasibility correction is critical for ML — the model itself does not enforce the capacity constraint.
4. For 0/1 Knapsack, DP remains the gold standard at moderate problem sizes.

Weighted Interval Scheduling

DP vs Greedy vs ML

A Comparative Analysis of Algorithmic Approaches

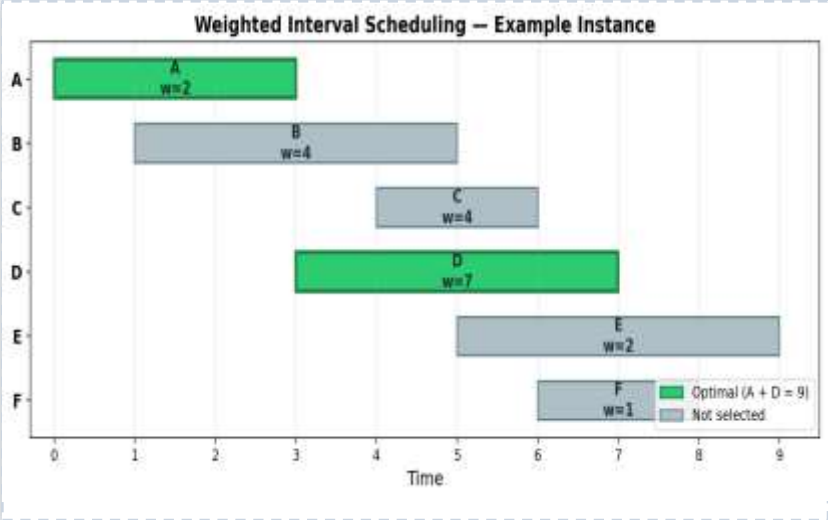
Fabricio Joel Gutierrez Juarez

Problem Statement

Given n jobs, each with a start time, end time, and weight (profit),
select a subset of non-overlapping jobs that maximizes total profit.

Example Instance

Job	Start	End	Weight
A	0	3	2
B	1	5	4
C	4	6	4
D	3	7	7
E	5	9	2
F	6	8	1



Optimal Answer: A + D = 9 (non-overlapping, highest total weight)

Dynamic Programming Approach

1. Sort jobs by finish time
2. Binary-search for latest non-conflicting job
3. dp[i] = max(dp[i-1], weight_i + dp[p(i)])
4. Traceback to find selected jobs

DP Table Walkthrough — Optimal: A + D = 9

i	Job	Interval	Weight	p(i)	dp[i-1]	w_i + dp[p(i)]	dp[i]	Choice
0	-	-	-	-	-	-	0	-
1	A	[0,3)	2	0	0	2+0=2	2	Include A
2	B	[1,5)	4	0	2	4+0=4	4	Include B
3	C	[4,6)	4	1	4	4+2=6	6	Include C
4	D	[3,7)	7	1	6	7+2=9	**9**	Include D
5	F	[6,8)	1	3	9	1+6=7	9	Skip F
6	E	[5,9)	2	3	9	2+6=8	9	Skip E

Traceback: dp[6]=9 → dp[5]=9 → dp[4]=9 (Include D) → dp[1]=p(D) → dp[1]=2 (Include A)

Complexity: O(n log n)

Always finds the optimal solution – our ground truth baseline

Greedy Approaches

Three heuristics — none guarantee optimality for the weighted case

Earliest Finish (EF)

Rule: Pick the job that ends soonest

Weakness: Ignores weights entirely

Highest Weight (HW)

Rule: Pick the heaviest available job

Weakness: "Baited" by one heavy job

Best Ratio (BR)

Rule: Pick by weight/duration ratio

Weakness: Can't reason globally

Example Walkthrough

EF: A → C → E/F (misses D's high weight)

HW: D first → nothing else fits well

DP: A + D = 9 (full picture)

ML Approach

1. Generate hundreds of random scheduling instances
2. Solve each with DP → labels (selected = 1, not = 0)
3. 16 features: weight, duration, ratio, conflicts, competitive ratio…
4. Train classifiers: Random Forest (200 trees) & MLP
5. Rank by confidence → greedily add non-conflicting jobs

Random Forest

200-tree ensemble. Good at feature thresholds.

MLP

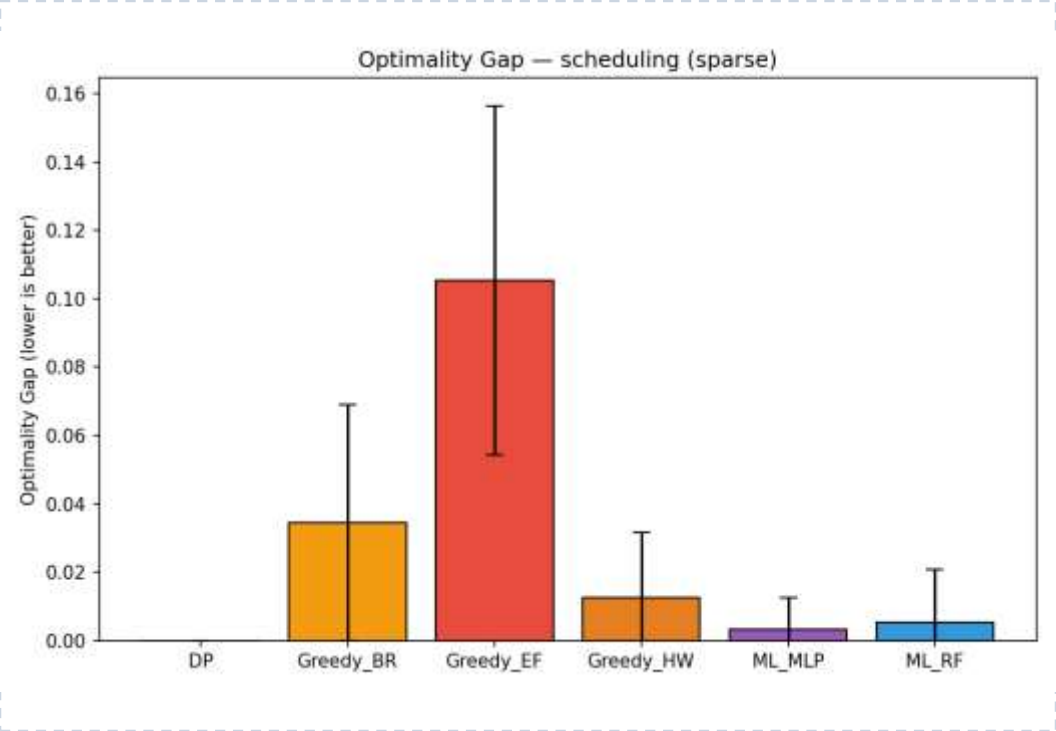
Small neural network. Learns nonlinear combinations.

Key Design Decisions

Class-balanced training, confidence-based greedy selection, 16 hand-crafted features including competitive ratio (weight vs. conflict weight).

Optional: Insert ML Pipeline Diagram

Results — Sparse Instances

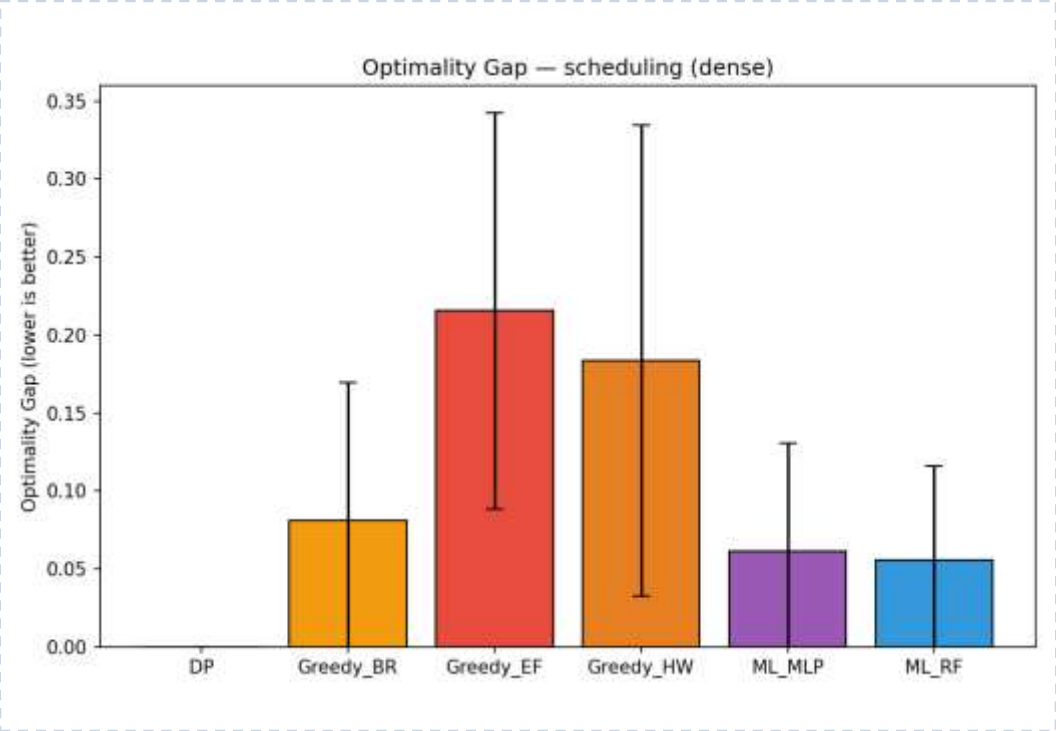


Key Findings

DP	**0%**	*always optimal*
ML MLP	**~0.4%**	*best non-DP*
ML RF	**~0.5%**	*best non-DP*
Greedy HW	**~1.3%**	*decent*
Greedy EF	**~10.5%**	*ignores weights*

Takeaway: ML approximates DP very closely (~0.4% gap) and outperforms all greedy heuristics on sparse instances.

Results — Dense Instances



Key Findings

ML RF	**~5.5%**	*best non-DP*
ML MLP	**~6%**	*best non-DP*
Greedy BR	**~8%**	*best greedy*
Greedy EF/HW	**18–22%**	*struggles*

Why ML Still Leads

Conflict-aware features (competitive ratio, conflict weight) give ML an edge over rigid greedy rules, even as all methods degrade on dense instances.

Takeaway: ML's learned features give it an edge over greedy even on hard instances, but none match DP's exact solution.

Results — Adversarial Cases

Adversarial construction: One long, very heavy job spanning the entire timeline, plus n short jobs packed inside that collectively outweigh it.

Method	Optimality Gap	What Happened
DP	0%	Finds optimal combination
Greedy EF	0%	Short jobs finish first — correct here
Greedy BR	0%	Short jobs have better ratio
Greedy HW	~17%	Falls for the bait — grabs heavy job
ML MLP	0%	Learned to favor high-ratio packed intervals
ML RF	0%	Learned pattern from training data

Takeaway: ML avoids the trap by considering multiple features (ratio, conflict weight), not just raw weight. Only Greedy HW falls for the bait. But adversarial cases targeting ML's features could break it too — only DP is universally robust.

Runtime Comparison

Method	n = 20	n = 50	Complexity
DP	~0.02 ms	~0.03 ms	O(n log n)
Greedy (all 3)	~0.01–0.02 ms	~0.02–0.04 ms	O(n log n)
ML MLP	~0.8 ms	~1.5 ms	$O(n) + O(n^2)$ feat.
ML RF	~40–60 ms	~50–70 ms	$O(n) + O(n^2)$ feat.

Why is ML the slowest?

Feature extraction (computing conflict counts/weights) is $O(n^2)$, and RF runs 200 trees per prediction. DP and Greedy are both O(n log n) and nearly instant at these sizes. ML's speed advantage only matters at much larger scales or when exact algorithms are exponential.

For problems where DP is polynomial, ML's speed advantage doesn't materialize. ML makes more sense for NP-hard problems at large scale.

Summary & Takeaways

	DP	Greedy	ML
Quality	Always optimal	OK on sparse, fails adversarial	Best non-DP on sparse and dense
Speed	Very fast O(n log n)	Very fast O(n log n)	Slowest (feature overhead)
Shines	Always	Quick approx. no training needed	Similar dist. to training
Fails	Never	Adversarial inputs	OOD inputs, runtime overhead

ML can learn to approximate DP surprisingly well, outperforming greedy.

But it adds complexity and runtime overhead not justified when DP already runs in microseconds.

The real value of ML for optimization is on NP-hard problems where exact algorithms are exponential.